

Proof Buddy — Research Context

1. What Is Proof Buddy?

Proof Buddy is an educational software platform designed to help students learn and practice formal mathematical proof techniques. It was developed for use in undergraduate computer science and mathematics courses at Drexel University. The system provides an interactive, computer-checked environment where students can construct proofs step-by-step and receive immediate feedback on whether each step is logically valid.

The central insight driving the project is that students learning formal proof — particularly equational reasoning and mathematical induction — benefit enormously from a tool that:

1. Enforces formal syntax so informal hand-waving cannot pass as a proof.
 2. Checks each step immediately, stopping errors before they compound.
 3. Allows students to experiment freely: trying a rule, seeing if it works, and undoing it if not.
 4. Makes the structure of a proof visible in a way that a sheet of paper cannot.
-

2. The Mathematical Language: Racket-Style S-Expressions

Proof Buddy uses a Racket-like syntax (parenthesized prefix notation) for all expressions. This is not arbitrary. Racket (a dialect of Scheme/Lisp) presents arithmetic and logic in a uniform, unambiguous tree structure that is ideal for formal reasoning:

- `(+ 2 3)` means $2 + 3$
- `(cons a L)` builds a list by prepending element `a` to list `L`
- `(if (null? L) 0 (+ 1 (length (rest L))))` is a conditional definition

This notation is already familiar to students in courses that use the textbook *How to Design Programs* (HtDP), which teaches functional programming in Racket. Proof Buddy leverages this familiarity to lower the notational barrier when introducing formal proof reasoning.

The expression syntax maps directly to an abstract syntax tree (AST), which Proof Buddy uses internally to track nodes, apply rules, and verify structural equality — this is how the proof engine works at a deep level.

3. Proof Types Supported

3.1 Equational Reasoning

Equational reasoning is the practice of proving that two expressions are equal by rewriting each side through a series of valid algebraic or logical transformations until both sides become the same expression.

Example goal: Prove that `(length (append L M)) = (+ (length L) (length M))`

The student works on the left-hand side (LHS) and right-hand side (RHS) in alternating steps (or in parallel), applying rewrite rules at each step. When the LHS and RHS reach identical expressions, the proof is complete.

This form of reasoning is fundamental in functional programming verification, algebra, and discrete mathematics. Students learning Racket-based programming encounter it when verifying that their function implementations satisfy algebraic laws.

Supported rules in equational reasoning:

- **eval**: Apply a function or rule at a specific node (e.g., evaluate `(+ 2 3) → 5`, or expand `(length '()) → 0` using the definition of length)
- **apply**: Apply a UDF definition at a call site
- **rewrite**: Rewrite using an axiom or known identity
- **rewrite IH**: Use the inductive hypothesis (in induction mode only)
- **rewrite math**: Invoke SymPy to symbolically simplify a mathematical expression

3.2 Mathematical Induction

Mathematical induction is a proof technique for statements of the form "for all n , $P(n)$ is true", where n ranges over natural numbers or lists. It requires:

1. **Base case**: Prove $P(\text{anchor_value})$, e.g., $P(0)$ or $P(\text{null})$.
2. **Leap step** (inductive step): Assume $P(k)$ (the inductive hypothesis), and prove $P(k+1)$ or $P(\text{cons } a \ K)$.

Proof Buddy supports both **integer induction** (over natural numbers, with anchor value 0 and leap from k to $k+1$) and **list induction** (over lists, with anchor value `null` and leap from K to `(cons a K)` for UP induction or K for DOWN induction from `(cons a K)`).

Why induction? Induction is the canonical proof technique for properties of recursive data structures and functions. Since the course context involves Racket programming, where students write recursive functions, proving properties of those functions via induction is a natural and important skill.

3.3 Natural Deduction (Legacy Support)

Natural deduction is a formal proof system for propositional and first-order logic. It includes rule forms like:

- **TFL (Truth-Functional Logic / Propositional Logic)**: Modus Ponens, Modus Tollens, Disjunctive Syllogism, Addition, etc.
- **FOL (First-Order Logic)**: Universal and existential quantifier introduction and elimination.

The `proofs` and `racket_api` apps, along with the `NaturalDeductionPropositionalLogic.js` and `NaturalDeductionFirstOrderLogic.js` frontend pages, implement this older mode. These pages predate the current equational/induction system and use different backend logic. They remain functional but are not the focus of active development.

4. How the Software Connects to Pedagogy

4.1 The Proof-as-Program Analogy

In courses at Drexel that use this tool, mathematical proof is taught alongside programming. Just as writing a program requires following syntactic and semantic rules, writing a formal proof requires following rules of inference. Proof Buddy enforces both kinds of rules computationally, giving students the same kind of immediate feedback they get from a compiler or interpreter — but for proofs.

4.2 The Role of User-Defined Functions (UDFs)

Students often prove properties of their own function definitions. The UDF mechanism in Proof Buddy lets students enter the same function they wrote in their programming assignment (in Racket syntax) and then use that definition as a rule in a proof. For example:

```
; Student's function definition:
(define (length L)
  (if (null? L) 0 (+ 1 (length (rest L)))))
```

In Proof Buddy, they enter this as a definition labeled `length`. Then in a proof, they can apply the rule "`eval length`" at any `(length ...)` call site, and the system will expand the definition correctly.

4.3 Generic Variables

Generic variables allow proofs to work over symbolic, unspecified values rather than concrete numbers. For example, to prove a property for all integers `n`, the student introduces `n` as a `GenericInt`. The proof engine treats it as a symbolic integer, and rules involving inequalities or arithmetic are checked symbolically (using SymPy where needed).

Generic variables have types (`int`, `list`, `bool`, `any`) and optional constraints (e.g., "`non-negative`", "`never-null`"). These constraints allow the proof engine to validate conditional rules correctly — for instance, knowing that a list generic is never null means `null?` checks on it simplify to `#f`.

4.4 Instructor Features

Proof Buddy supports classroom use where instructors:

- Create **assignments** tied to a **term** (course section).
- Assign proof problems to students.
- Review student submissions with grades.
- Create **partial proofs** with some lines hidden (`hide_expression`, `hide_justification`) as scaffolded exercises.
- Monitor which proofs are complete vs. incomplete.

The `is_instructor` flag on user accounts gates access to these features. Students cannot see hidden proof lines until after they have filled in those steps themselves.

5. Types of Formal Logic the System Understands

Logic Type	Mode	Status
Equational reasoning over integers and lists	Equational Reasoning (new)	Active, fully implemented
Mathematical induction over integers	Induction (new)	Active, fully implemented

Logic Type	Mode	Status
Mathematical induction over lists (UP)	Induction (new)	Partially implemented (in progress)
Mathematical induction over lists (DOWN)	Induction (new)	Planned, not yet implemented
Truth-Functional Logic (TFL / propositional)	Natural Deduction (legacy)	Functional but legacy
First-Order Logic (FOL)	Natural Deduction (legacy)	Functional but legacy

The current active development focus is on equational reasoning and induction with the new architecture (the [equational_reasoning_api](#) and [induction_api](#) Django apps backed by the [expression_tree](#) proof engine).

6. The Research Goals of the Project

Proof Buddy is a research project as well as a teaching tool. Its broader goals include:

6.1 Studying How Students Learn Formal Proof

By logging every proof step a student makes — including failed attempts, deleted lines, and rule applications with errors — the system creates a detailed record of the student's reasoning process. This data can be mined to study:

- Common misconceptions about rule application
- Which rules students find hardest to apply correctly
- How students structure their proof search (top-down vs. bottom-up)
- Whether having immediate feedback improves proof quality compared to paper-and-pencil proofs

6.2 Validating the Pedagogical Effectiveness of Racket-Syntax Proofs

One research question underlying the project is whether using a programming-language-like syntax for proofs (Racket *s*-expressions) helps computer science students better understand the connection between programming and mathematical reasoning. The tool being used in actual CS courses at Drexel University provides a natural experimental context for this question.

6.3 Supporting Scalable Grading

For courses with many students, hand-grading formal proofs is labor-intensive. Proof Buddy's automated checking allows instructors to verify correctness instantly, reducing grading burden while also giving students better feedback.

6.4 Developing Proof Automation Research

The proof engine's rule-based architecture makes it a candidate for future work on automated proof assistance: hint generation, incomplete-proof detection, and guided proof search. The generic variable system

and symbolic math integration (via SymPy) lay groundwork for more sophisticated reasoning support.

7. Classroom Studies and Experiments

Proof Buddy has been deployed in Drexel University CS courses that teach discrete mathematics and functional programming. Typical use cases include:

1. **Equational proof assignments:** Students prove algebraic identities involving `length`, `append`, or arithmetic expressions. The proof is submitted through the system, which checks it automatically and records it to the student's account.
 2. **Induction proofs:** Students prove by induction that recursive function definitions satisfy stated properties (e.g., that $(\text{length } (\text{append } L \ M)) = (+ \ (\text{length } L) \ (\text{length } M))$).
 3. **In-class demonstration:** Instructors can demonstrate proof construction live, with the system projecting the step-by-step proof to students and showing what happens when a student tries an invalid step.
 4. **Scaffolded exercises:** Instructors use the `hide_expression` / `hide_justification` feature to provide partially-completed proofs where students fill in missing steps.
-

8. Connection to HtDP and Functional Programming Education

The project sits at the intersection of two established educational traditions:

1. **How to Design Programs (HtDP):** A curriculum that teaches programming systematically by connecting code structure to data structure and recursive definitions to inductive reasoning. Proof Buddy makes the inductive reasoning aspect explicit and formal.
2. **Formal Methods in CS Education:** A larger movement to bring formal verification tools into undergraduate education, making the idea that "software can be proven correct" accessible to students before they encounter industrial verification tools like Coq or Isabelle.

Proof Buddy occupies a middle ground: more formal than pencil-and-paper homework, but less intimidating than industrial proof assistants. This makes it an effective bridge tool for students who will go on to formal software verification work.